A Multi-Agent System to Repair Breaking Changes Caused by Dependency Updates in Kotlin Multiplatform

Santiago Bobadilla Suarez
s.bobadilla@uniandes.edu.co
Universidad de los Andes
Bogotá, Colombia

Camilo A. Escobar Velasquez
ca.escobar2434@uniandes.edu.co
Universidad de los Andes
Bogotá, Colombia

1. Introduction

Modern software systems increasingly rely on open-source software libraries distributed via package managers, which applications include as versioned dependencies. [4] Keeping dependencies up to date is essential to obtain security patches, performance improvements, and new features. [2] In practice, however, upgrades frequently introduce breaking changes, such as removed APIs or altered type signatures, that disrupt clients at compile time and, in some cases, at runtime. Empirical studies on Maven Central report that roughly 30 % of releases contain at least one breaking change. [5]

Dependency management is further complicated by limited supply-chain visibility. High-profile incidents (e.g., Log4J) have shown that organizations frequently lack an accurate inventory of third-party components and how they enter their systems. [4] Moreover, vulnerabilities commonly arise not only through direct dependencies but also through transitive dependencies that developers neither explicitly declare nor routinely review. [2] This opacity slows vulnerability response and long-term maintenance, contributing to technical lag, where packages remain behind newer releases and staleness correlates with exposure to known vulnerabilities. [4]

Automated dependency-update bots such as Dependabot and Renovate handle the mechanical part of upgrades by opening pull requests that bump versions. [1] The harder part still falls on developers and reviewers: they must interpret release notes, map API, and behavioral changes to their own call sites, and understand why a build or test started failing. A version-only diff may be small, even redundant, but it still needs a risk assessment. [1, 2] As a result, many tools behave more like notification systems than debugging aids: they surface available upgrades and advisories, but they rarely pinpoint downstream impact, explain failures when tests are incomplete, or propose concrete patches when compilation or tests break. [2, 4]

This paper addresses this gap by proposing an AI-agent-based workflow that applies dependency upgrades, localizes and repairs resulting breakages, and produces human-readable explanations of what changed and why. We scope our approach to Kotlin Multiplatform Mobile (KMM) Mono-Repos, where Kotlin code is shared across Android and iOS while user interfaces remain native. [3] KMM projects use platform-aware source sets (e.g., commonMain, androidMain, iosMain) and cross-platform contracts via Kotlin's expect/actual mechanism. A dependency change affecting shared code can therefore break multiple source sets, and failures may be non-trivial to attribute. [3]

These characteristics make KMM a particularly challenging setting for dependency maintenance, where breakages may span multiple targets and their root cause is not always obvious. In this context, we aim to reduce the manual effort required to keep KMM dependencies current by automatically identifying the downstream impact of upgrades and generating actionable fixes. We evaluate whether an AI-agent-based pipeline can (i) apply upgrades, (ii) localize and repair resulting failures, and (iii) generate human-readable rationales that aid reviewers. We present the system, discuss its limitations, and evaluate its performance on representative KMM upgrade scenarios.

Referencias

[1] Filipe R. Cogo and Ahmed E. Hassan. 2022. Understanding the Customization of Dependency Bots: The Case of Dependabot. *IEEE Software* 39, 5 (2022), 44–49. doi:10.1109/MS.2022.3179484

[2] Joseph Hejderup and Georgios Gousios. 2022. Can we trust tests to automate dependency updates? A case study of Java Projects. *Journal of Systems and Software* 183 (2022), 111097. doi:10.1016/j.jss.2021.111097

[3] Manishankar Janakaraj. 2025. Architecting Shared Logic with Kotlin Multiplatform Mobile: Benefits, Challenges, and Ecosystem Positioning. *European Journal of Computer Science and Information Technology* 13, 44 (June 2025), 121–141. doi:10.37745/ejcsit.2013/vol13n44121141 Published June 26, 2025. Print ISSN: 2054-0957; Online ISSN: 2054-0965.

[4] Tom Mens and Alexandre Decan. 2024. An Overview and Catalogue of Dependency Challenges in Open Source Software Package Registries. In *Proceedings of the 23nd Belgium-Netherlands Software Evolution Workshop, Namur, Belgium, November 21-22, 2024 (CEUR Workshop Proceedings, Vol. 3941)*, Gilles Perrouin, Benoît Vanderose, and Xavier Devroey (Eds.). CEUR-WS.org, 160–176. https://ceur-ws.org/Vol-3941/BENEVOL2024_TECH_paper15.pdf

[5] S. Raemaekers, A. van Deursen, and J. Visser. 2017. Semantic versioning and impact of breaking changes in the Maven repository. *Journal of Systems and Software* 129 (2017), 140–158. doi:10.1016/j.jss.2016.04.008